A AgriProvider

POSTMAN COLLECTION GUIDE

API Requests in Postman

Setup instructions and endpoint reference for the AgriProvider Postman collection.

Version 1 · 10 November 2026 · API base URL: `http://localhost:5000`

Import these files

`AgriProvider_API.postman_collection.json`
`AgriProvider_Local.postman_environment.json`

The collection contains 53 requests grouped by health/authentication, provider workflows, assessor review, administrator assignments, and the learner portal. The environment has blank credential and token fields; no secrets are included.

This guide describes implemented API behavior only. It does not imply official QA, certification, eligibility, or payment functionality.

Quick setup

1. Start the API from `server/` using `npm start`. Confirm `GET /health` returns a connected database.
2. In Postman, choose **Import** and import both JSON files from the repository `docs/` folder.
3. Select the **AgriProvider Local** environment in the top-right environment menu.
4. Enter test values into the environment's `email`, `password`, and, for provider registration, `fullName`. Do not save real secrets in a shared export.
5. Send **Health and Authentication** → **Login**, or **Register provider** for a new provider account.
6. On successful login/registration, the collection test script stores `accessToken`, `userId`, and `userRole` in the selected environment. Other requests use the saved bearer token.
7. Create resources in order. Successful create requests save their IDs, such as `programmeId`, `learnerId`, `enrolmentId`, `sessionId`, and `requirementId`.

> **Requests can change data.** Use an approved development database with synthetic records. Do not point mutating requests at production without explicit approval.

Authentication behavior

The collection uses bearer-token authentication for protected requests. Login and register are configured as public requests. The server issues a one-hour JWT for native/Postman usage. A successful account role is read from the database, not granted by a Postman variable.

Public registration creates **provider accounts only**. Learner, assessor, QA, and admin accounts must be provisioned through the approved server process. A learner login also needs an explicit or supported legacy profile link.

> **Web-cookie flow:** browser sessions use an HttpOnly cookie, exact `WEB_ORIGIN`, `X-Auth-Mode: cookie`, and credentialed requests. Postman examples use bearer tokens; the full cookie flow is covered in the API reference PDF.

Environment variables

`baseUrl` defaults to `http://localhost:5000`. The environment also defines credentials, token, resource IDs, status filters, and request fields. Empty optional ID filters are omitted from list requests to avoid invalid UUID errors.

Postman request catalog

Health and authentication

Method	Path	Role and behavior
GET	`/health`	Public database health check.
POST	`/api/auth/register`	Public provider registration; returns user/token (bearer clients).
POST	`/api/auth/login`	Public sign-in; optional accountType validates but never changes stored role.
GET	`/api/auth/me`	Authenticated current-user profile.
POST	`/api/auth/logout`	Clears web cookie; bearer clients discard token locally.

Provider programmes, learners, enrolments

Method	Path	Role and behavior
GET / POST	`/api/programmes`	Provider: list active programmes / create with name and optional description.
GET / PATCH / DELETE	`/api/programmes/:id`	Provider: retrieve, update, or archive an owned programme.
GET	`/api/learners`	Provider: search/paginate; query supports q, status, limit, offset.
POST	`/api/learners`	Provider: create profile; possible duplicate returns 409 and confirmation code.
GET / PATCH / DELETE	`/api/learners/:id`	Provider: retrieve, update, or archive an owned learner; archive withdraws active enrolments.
GET	`/api/enrolments`	Provider: list with optional status, learnerId, programmeId, limit, offset filters.
POST	`/api/enrolments`	Provider: create same-provider enrolment using learnerId and programmeId.
GET	`/api/enrolments/:id`	Provider: retrieve owned enrolment.
PATCH	`/api/enrolments/:id`	Provider: set status to active or withdrawn.

Sessions and attendance

Method	Path	Role and behavior
GET / POST	`/api/sessions`	Provider: list (optional programmeId) / create with programmeId, title, timezone-bearing startsAt.
GET / PATCH / DELETE	`/api/sessions/:id`	Provider: retrieve, update, or archive session.
GET	`/api/sessions/:id/attendance`	Provider: active enrolment register and status summary.

Method	Path	Role and behavior
PUT	`/api/sessions/:id/attendance`	Provider: atomically upsert records; status present, absent, or excused.

Evidence and review catalog

Evidence requirements and submissions · provider

Method	Path	Behavior
GET / POST	`/api/programmes/:programmeId/evidence-requirements`	List or create configured checklist items.
GET / PATCH / DELETE	`/api/programmes/:programmeId/evidence-requirements/:id`	Retrieve, update, or archive a requirement.
POST	`/api/enrolments/:enrolmentId/evidence-requirements/:requirementId/submissions`	Multipart field `file`; PDF/JPEG/PNG ≤5 MB. New upload starts SUBMITTED.
GET	`/api/enrolments/:enrolmentId/evidence-submissions`	Checklist, latest versions, reviews, and completion counts.
GET	`/api/evidence-submissions/:submissionId`	Owned submission metadata.
GET	`/api/evidence-submissions/:submissionId/history`	All versions and associated reviews.
GET	`/api/evidence-submissions/:submissionId/file`	Private authorized file download.
DELETE	`/api/evidence-submissions/:submissionId`	Only latest unreviewed SUBMITTED version can be deleted.

Assessor review · active assignment required

Method	Path	Behavior
GET	`/api/assessor/programmes`	Assigned programmes and counts.
GET	`/api/assessor/programmes/:programmeId/learners`	Active enrolled learners.
GET	`/api/assessor/programmes/:programmeId/submissions`	Latest submissions; filter with status query.
GET	`/api/assessor/submissions/:submissionId`	Assigned submission details.
GET	`/api/assessor/submissions/:submissionId/file`	Private assigned evidence download.
GET	`/api/assessor/submissions/:submissionId/history`	All versions for assigned requirement.
GET	`/api/assessor/submissions/:submissionId/reviews`	Version-bound review history.
POST	`/api/assessor/submissions/:submissionId/reviews`	APPROVED or CORRECTION_REQUESTED; correction requires feedback. Latest SUBMITTED version only.

> Review decisions are immutable and tied to the exact evidence version and assignment. Provider resubmission is correction-gated.

Admin, learner, and troubleshooting

Administrator assignments

Method	Path	Behavior
GET	`/api/admin/assessors`	List active assessors.
GET	`/api/admin/programmes`	List active programmes.
GET	`/api/admin/assessor-assignments`	Assignment history.
POST	`/api/admin/assessor-assignments`	Create using programmeId and assessorUserId UUIDs.
PATCH	`/api/admin/assessor-assignments/:assignmentId`	Set ACTIVE/INACTIVE or reassign using assessorUserId.

Learner portal

Method	Path	Behavior
GET	`/api/learner/dashboard`	Learner-only. Own linked profile, programmes, attendance, evidence versions, learner-visible feedback, counts, recent activity.
GET	`/api/learner/programmes`	List the caller's linked programmes.
GET	`/api/learner/programmes/:id`	Retrieve only a programme linked to the caller.

Common responses

400 Invalid request **401 Sign-in required/rejected** **403 Role/origin denied** **404 Not found/in scope** **409 Conflict/state**

413 Too large **429 Rate limited** **500 Internal error** **503 Service unavailable**

- 401 on login: check email/password and account active status. Error intentionally does not confirm whether an email exists.
- 403: verify the signed-in role and, for assessors, active assignment.
- 404: confirm resource IDs and that the caller owns or is assigned to the resource.
- 409: check duplicate, workflow state, or resubmission requirements.
- 503: check API/database connectivity and configured storage.

> **Not available through this API:** learner uploads, notifications/announcements, QA decisions, official certification, eligibility calculations, payment execution, and external AgriSETA/government integration.

For full field constraints, authentication cookie configuration, response details, and setup cautions, see `AgriProvider_API_Documentation.pdf` and `server/README.md`. API tests use an in-memory SQL stub; do not treat them as production database verification.